

Jenkins-04 — Task 1: Freestyle CI/CD for Hiring App

1) Setup Jenkins CICD pipeline using freestyle job using Docker containers using below code.

Source Code: <https://github.com/betawins/hiring-app.git>

Stages:

- Git Clone.
- SonarQube Integration.
- Maven Compilation.
- Nexus Artifactory.
- Slack Notification.
- Deploy On tomcat.

Jenkins Hiring App CI/CD Pipeline

Git → Maven → SonarQube → Nexus → Tomcat → Slack

1. Objective

Configure a Jenkins Freestyle Job that will:

1. Checkout source code from GitHub.
2. Build the application using Maven and generate a WAR file.
3. Perform code analysis using SonarQube.
4. Upload WAR and POM artifacts to Nexus Repository Manager.
5. Deploy the WAR file to a Tomcat Docker container.
6. Send build notifications to Slack.

Pipeline Flow

Plain Text

GitHub Repository

↓

Maven Build (WAR)

↓

SonarQube Analysis

↓

Upload Artifacts to Nexus

↓

Deploy to Tomcat Docker

↓

Slack Notification

Show more lines

2. Lab Infrastructure

Component	Private IP	Purpose
Jenkins Server	172.31.12.117	Build & Deployment Server
SonarQube Server	172.31.47.228	Code Quality Analysis
Nexus Server	172.31.15.170	Artifact Repository
Tomcat Container	Jenkins Server	Application Hosting
Slack Workspace	Existing Workspace	Build Notifications

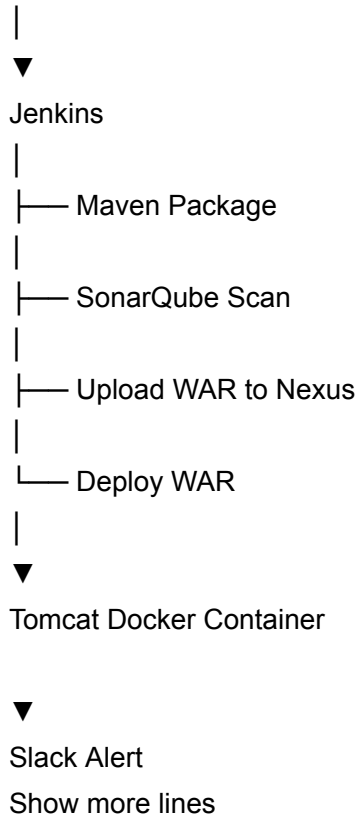
Access URLs

Service	URL
Jenkins	http://44.223.94.171:8081
SonarQube	http://172.31.47.228:9000
Nexus	http://44.212.46.67:8082
Hiring App	http://44.223.94.171:8083/hiring/

3. Architecture Overview

Plain Text

GitHub Repository



4. Prerequisites

The following components were already available from previous labs:

- Jenkins
- Java
- Maven
- Git
- SonarQube
- Slack Integration

New components created in this lab:

- Nexus Repository Server
 - Tomcat Docker Container
-

5. Setup Nexus Repository Manager

Server

Plain Text

172.31.15.170

Show more lines

Install Docker

Shell

```
dnf install -y docker
```

```
systemctl enable --now docker
```

```
docker --version
```

Show more lines

Create Nexus Container

Shell

```
docker volume create nexus-data
```

```
docker run -d \
```

```
--name nexus \
```

```
--restart unless-stopped \
```

```
-p 8082:8081 \
```

```
-v nexus-data:/nexus-data \
```

```
sonatype/nexus3:latest
```

Show more lines

Verify Container

Shell

```
docker ps
```

```
docker logs nexus
```

```
curl http://localhost:8082
```

Show more lines

Access Nexus

Plain Text

http://44.212.46.67:8082

Show more lines

6. Create Nexus Repository

Navigate:

Plain Text

Nexus

→ Settings

→ Repositories

→ Create Repository

→ Maven2 Hosted

Show more lines

Repository Details:

Setting	Value
Name	hiring-app
Type	Maven Hosted
Deployment Policy	Allow Redeploy
Online	Enabled

Repository URL:

Plain Text

http://172.31.15.170:8082/repository/hiring-app/

Show more lines

7. Create Nexus Deployment User

Navigate:

Plain Text

Security

→ Roles

→ Create Role

Show more lines

Role Name:

Plain Text

hiring-app-deployer

Show more lines

Assign Privileges:

Plain Text

nx-repository-view-maven2-hiring-app-add

nx-repository-view-maven2-hiring-app-edit

nx-repository-view-maven2-hiring-app-read

nx-repository-view-maven2-hiring-app-browse

Show more lines

Create User:

Plain Text

jenkins-deployer

..

Show more lines

Assign Role:

Plain Text

hiring-app-deployer

Show more lines

Test Access:

Shell

curl --user jenkins-deployer \

http://172.31.15.170:8082/repository/hiring-app/

Show more lines

Expected:

Plain Text

HTTP 200

Show more lines

8. Create Jenkins Freestyle Job

Job Name

Plain Text

Jenkins-04-Hiring-App

Show more lines

Source Code Management

Select:

Plain Text

Git

Show more lines

Repository:

Plain Text

<https://github.com/betawins/hiring-app.git>

Show more lines

Branch:

Plain Text

*/main

Show more lines

Execution Node:

Plain Text

built-in

Show more lines

9. Build Application Using Maven

Add Build Step:

Plain Text

Execute Shell

Show more lines

Script:

Shell

set -e

```
mvn -B clean package \
-Dmaven.compiler.source=8 \
-Dmaven.compiler.target=8
```

ls -lh target/hiring.war

Show more lines

Expected Result:

Plain Text

BUILD SUCCESS

target/hiring.war created

Show more lines

Generated Artifact:

Plain Text

target/hiring.war

Show more lines

10. Configure SonarQube Analysis

Verify Connectivity

Shell

```
curl \
```

```
http://172.31.47.228:9000/api/system/status
```

Show more lines

Expected:

Plain Text

UP

Show more lines

Jenkins Configuration

Plain Text

Manage Jenkins

→ System

→ SonarQube Servers

Show more lines

Configure:

Field	Value
Name	SonarQube-Server
URL	http://172.31.47.228:9000
Token	Existing Token

Add SonarQube Build Step

.properties

sonar.projectKey=Jenkins-04-Hiring-App

sonar.projectName=Jenkins-04-Hiring-App

sonar.projectVersion=0.1

sonar.sources=src

sonar.sourceEncoding=UTF-8

Show more lines

Expected Output:

Plain Text

ANALYSIS SUCCESSFUL

EXECUTION SUCCESS

Show more lines

11. Configure Nexus Credentials in Jenkins

Navigate:

Plain Text

Manage Jenkins

→ Credentials

→ Global Credentials

→ Add Credentials

Show more lines

Type:

Plain Text

Username with Password

Show more lines

Store:

Plain Text

ID = nexus-credentials

Show more lines

Bind Variables:

Plain Text

NEXUS_USER

NEXUS_PASS

Show more lines

12. Upload Artifacts to Nexus

Add Build Step after SonarQube.

Shell

set +x

set -eu

```
NEXUS_PATH="http://172.31.15.170:8082/repository/hiring-app/in/javahome/hiring/0.1"
```

```
curl --fail \
```

```
--user "NEXUS_USER:NEXUS_PASS" \
```

```
--upload-file target/hiring.war \
```

```
"$NEXUS_PATH/hiring-0.1.war"
```

```
curl --fail \
```

```
--user "NEXUS_USER:NEXUS_PASS" \
```

```
--upload-file pom.xml \
```

```
"$NEXUS_PATH/hiring-0.1.pom"
```

```
echo "Upload Successful"
```

Show more lines

Verify Artifact in Nexus:

Plain Text

Browse

→ hiring-app

→ in/javahome/hiring/0.1

Show more lines

Expected:

Plain Text

hiring-0.1.war
hiring-0.1.pom
[Show more lines](#)

13. Enable Jenkins Docker Access

Run on Jenkins Server:

Shell
usermod -aG docker jenkins

systemctl restart jenkins
[Show more lines](#)

Verify:

Shell
sudo -u jenkins docker ps
[Show more lines](#)

Expected:

Plain Text
Docker command runs successfully
[Show more lines](#)

14. Deploy Tomcat in Docker

Create Container:

Shell
docker run -d \
--name tomcat-hiring \
--restart unless-stopped \
-p 8083:8080 \

```
-v tomcat-hiring-webapps:/usr/local/tomcat/webapps \
tomcat:9.0-jdk17-temurin
Show more lines
```

Verify:

```
Shell
docker ps
docker logs tomcat-hiring
Show more lines
```

Expected:

```
Plain Text
Server startup in ...
Show more lines
```

15. Automated Deployment to Tomcat

Add Build Step after Nexus Upload.

```
Shell
set -eu

docker stop tomcat-hiring

docker cp target/hiring.war \
tomcat-hiring:/usr/local/tomcat/webapps/hiring.war

docker start tomcat-hiring

for attempt in $(seq 1 30)
do
STATUS=$(curl -s -o /dev/null \
-w '%{http_code}' \
http://localhost:8083/hiring/ || true)
```

```
if [ "$STATUS" = "200" ]; then  
echo "Application Deployed Successfully"  
exit 0  
fi
```

```
sleep 2  
done
```

```
exit 1  
Show more lines
```

Expected:

```
Plain Text  
STATUS=200  
Application Deployed Successfully  
Show more lines
```

16. Configure Slack Notifications

Navigate:

```
Plain Text  
Job Configuration  
→ Post Build Actions  
→ Slack Notifications  
Show more lines
```

Enable:

☒ Notify Success

☒ Notify Failure

Expected Message:

```
Plain Text  
Jenkins-04-Hiring-App
```

#7 Success

Show more lines

17. Verify Complete CI/CD Pipeline

Click:

Plain Text

Build Now

Show more lines

Verify the following stages execute successfully:

✓ Git Checkout

✓ Maven Build

✓ SonarQube Scan

✓ Nexus Upload

✓ Tomcat Deployment

✓ Slack Notification

Console Output:

Plain Text

BUILD SUCCESS

ANALYSIS SUCCESSFUL

WAR and POM uploaded to Nexus successfully

SUCCESS: Hiring application deployed to Tomcat

Finished: SUCCESS

Show more lines

18. Application Verification

Open:

Plain Text

<http://44.223.94.171:8083/hiring/>

Show more lines

Expected Result:

Plain Text

Welcome to Techie Horizon

Show more lines

HTTP Response:

Plain Text

200 OK

Show more lines

19. Troubleshooting Summary

Issue	Resolution
Git URL Invalid	Use repository URL only
Nexus 401 Error	Use correct credentials
Jenkins Docker Permission Denied	Add Jenkins user to Docker group
Tomcat HTTP 404	Access /hiring/ context path
Deployment Returned 000	Wait for Tomcat startup

20. Final Result

Successfully implemented an end-to-end Jenkins Freestyle CI/CD pipeline for the Hiring App.

Workflow Executed

Plain Text

GitHub

↓

Jenkins Checkout

↓

Maven Build

↓

SonarQube Analysis

↓

Nexus Artifact Storage

↓

Tomcat Deployment

↓

Slack Notification

Show more lines

Outcome

✓ Application built successfully

✓ SonarQube analysis completed

✓ WAR and POM uploaded to Nexus

✓ Application deployed to Tomcat

✓ Slack notifications delivered

✓ Application accessible at:

<http://44.223.94.171:8083/hiring/>

sonatype nexus repository
Community Edition

Browse
Browse my repositories

Connect
Connect to a repository

Release Notes
See what's new

Notification Center
You have 1 notification to review

Nexus Repository Cloud Migrator Is Now Available
Easily migrate your Nexus instance to the cloud without downtime. [Learn More](#)

11th Annual SSCR
[DOWNLOAD NOW](#)

Sonatype Nexus Repository
Community Edition

Welcome to Sonatype Nexus Repository

24 of 24 - Clipboard
Item not Collected: Format not supported by Office Clipboard

sonatype nexus repository
Community Edition

Search components or CVEs...

Usage Center Usage below limits

Monitor this instance's usage to ensure your deployment is appropriate for your needs. [Learn more about the usage center](#)

Usage Metrics Overview

Total Components	Requests Per Day	Requests Per Month
0 Current 40,000 Usage Limit	0 Last 24 hours 100,000 Usage Limit	0 Total requests in October 0 Average requests (12 months) 0 Highest recorded count (12 months)

System Health Check
View system status checks

Cleanup Policies
Review component removal policies

Browse
Browse my repositories

Notification Center
You have 1 notification to review

sonatype nexus repository Community Edition

Repositories

Create repository

Name	Type	Format	Blob Store	Status	URL	Health check	Firewall Re...
hiring-app	hosted	maven2	default	Online	copy		
maven-central	proxy	maven2	default	Online - Ready to ...	copy	0 0	
maven-public	group	maven2	default	Online	copy		
maven-releases	hosted	maven2	default	Online	copy		
maven-snapshots	hosted	maven2	default	Online	copy		
nuget-group	group	nuget	default	Online	copy		
nuget-hosted	hosted	nuget	default	Online	copy		
nuget.org-proxy	proxy	nuget	default	Online - Ready to ...	copy	0 0	

sonatype nexus repository Community Edition

Users / Create User

psschandrashekar@gmail.com

Password:

Confirm password:

Status: Active

Roles:

Available

Filter

nx-admin

nx-anonymous

Granted

hiring-app-deployer

Create local user Cancel

44.223.94.171:8081/job/Jenkins-04-Hiring-App/

Jenkins / Jenkins-04-Hiring-App

Status

Changes

Workspace

Build Now

Delete Project

Configure

Rename

SonarQube

Jenkins-04-Hiring-App Passed

Hiring App CI/CD: Git checkout, SonarQube analysis, Maven build, Nexus artifact upload, Slack notification, and Tomcat deployment using Docker containers.

SonarQube Quality Gate

Jenkins-04-Hiring-App Passed

server-side processing: Success

Permalinks

- Last build (#6), 3 min 46 sec ago
- Last stable build (#6), 3 min 46 sec ago
- Last successful build (#6), 3 min 46 sec ago
- Last completed build (#6), 3 min 46 sec ago

Builds

Filter

Today

- #7 4:57 PM
- #6 4:53 PM
- #4 4:08 PM
- #3 3:59 PM
- #2 3:58 PM

app.slack.com/client/T0C6VKQ6URE/C0C6VL57G2U

Chandrashekar-DevOps

#jenkins-alerts

Messages

@Jenkins is already in this channel.

Thursday, October 1st

Jenkins APP 9:50 PM
Slack/Jenkins plugin: you're all set on <http://32.196.117.219:8081/>

Jenkins APP 10:05 PM
Jenkins Job FAILED: Jenkins-03-VProfile #9

Jenkins APP 10:50 PM
Jenkins Job SUCCESS: Jenkins-03-VProfile #10

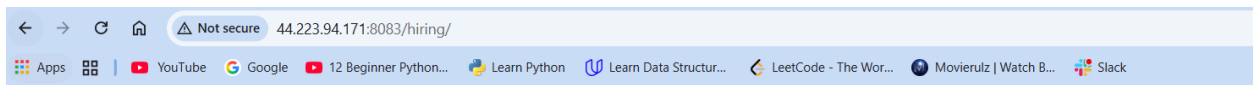
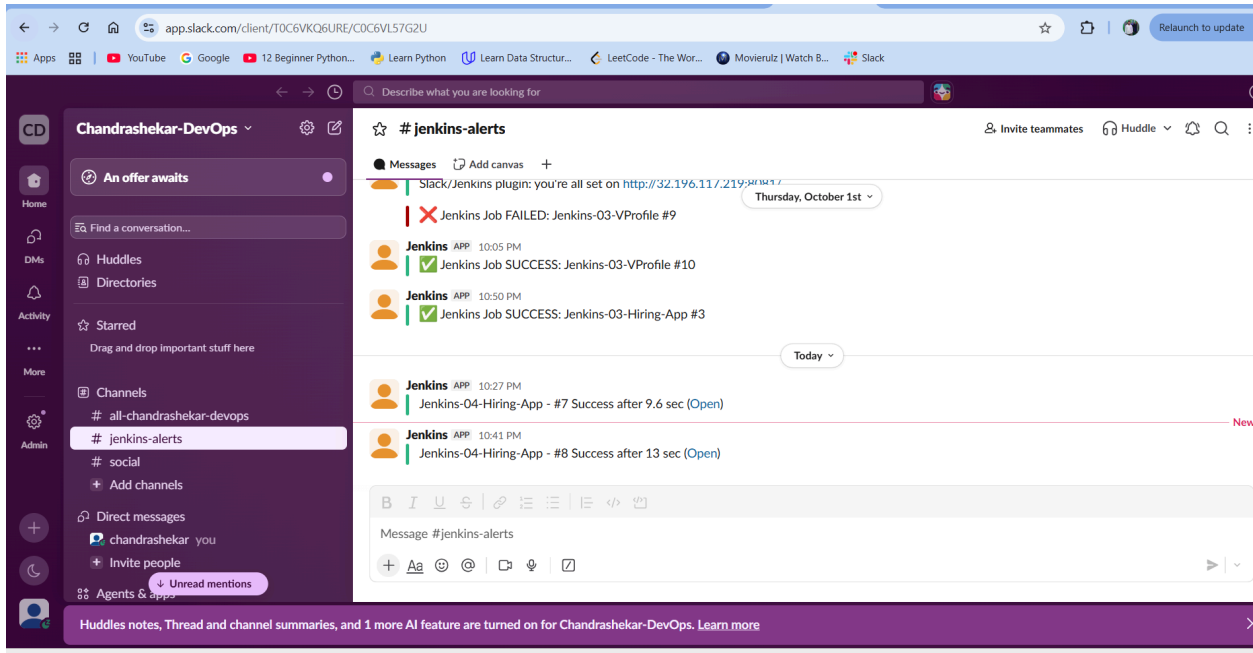
Jenkins APP 10:50 PM
Jenkins Job SUCCESS: Jenkins-03-Hiring-App #3

Today

Jenkins APP 10:27 PM
Jenkins-04-Hiring-App - #7 Success after 9.6 sec (Open)

Message #jenkins-alerts

Huddles notes, Thread and channel summaries, and 1 more AI feature are turned on for Chandrashekar-DevOps. Learn more



Welcome to Techie Horizon

2) Setup a Jenkins CI/CD pipeline using Declarative pipeline using feature-1.1 branch.

Source Code:

https://github.com/betawins/sabear_simplecutomerapp/tree/feature-1.1

stages:

- Git Clone.**
- SonarQube Integration.**
- Maven Compilation.**
- Nexus Artifactory.**
- Slack Notification.**
- Deploy On tomcat.**

Jenkins-04 — Task 2 Documentation

Deploy SimpleCustomerApp Using Jenkins Pipeline

Objective

The purpose of this task was to create a Jenkins CI/CD pipeline that automatically:

GitHub → SonarQube → Build Application → Nexus → Tomcat → Health Check → Slack

The application used in this task was:

SimpleCustomerApp

GitHub repository:

https://github.com/betawins/sabear_simplecutomerapp.git

Branch:

feature-1.1

1. Infrastructure Used

Component	Purpose
Jenkins	CI/CD automation
GitHub	Source code repository
SonarQube	Static code quality analysis
Ant	Build tool used to create WAR
Nexus	Artifact repository
Docker	Runs Tomcat container
Tomcat	Application server

Slack Build success/failure notifications

Ports used:

Jenkins : 8081
Nexus : 8082
Hiring App Tomcat : 8083
SimpleCustomerApp : 8084
SonarQube : 9000

2. Jenkins Pipeline Flow

The final pipeline flow was:

Developer Code
↓
GitHub
↓
Jenkins Git Clone
↓
SonarQube Analysis
↓
Ant Build
↓
SimpleCustomerApp.war
↓
Nexus Repository
↓
Tomcat Docker Container
↓
Application Health Check
↓
Slack Notification

3. Git Clone Stage

Jenkins clones the application from GitHub.

```
stage('Git Clone') {
```

```

steps {

    git branch: 'feature-1.1',
        url: 'https://github.com/betawins/sabear_simplecutomerapp.git'
}
}

```

Purpose

This stage gets the latest source code from GitHub into the Jenkins workspace.

Workspace:

/var/lib/jenkins/workspace/Task-02-SimpleCustomerApp-Pipeline

4. SonarQube Analysis

SonarQube was used to analyze Java code quality.

```

stage('SonarQube Analysis') {

    steps {

        script {

            def scannerHome = tool 'SonarScanner'

            withSonarQubeEnv('SonarQube-Server') {

                sh """
                ${scannerHome}/bin/sonar-scanner \
                -Dsonar.projectKey=SimpleCustomerApp \
                -Dsonar.projectName=SimpleCustomerApp \
                -Dsonar.sources=src
                """
            }
        }
    }
}

```

Purpose

SonarQube checks the application source code for:

- Bugs
- Code smells
- Security issues
- Maintainability problems
- Duplicate code

Successful output:

ANALYSIS SUCCESSFUL
EXECUTION SUCCESS

5. Initial Maven Build Problem

Initially we used:

```
mvn clean package
```

Maven showed:

No sources to compile
No tests to run

The generated WAR was only:

1.4 KB

When we inspected it:

```
jar tf target/*.war
```

it contained only:

```
META-INF/  
WEB-INF/  
WEB-INF/classes/  
META-INF/maven/
```

There were no actual application files.

Why?

The project uses an older directory structure:

src/
WebContent/

But standard Maven normally expects:

src/main/java/
src/main/webapp/

Therefore Maven created an almost empty WAR.

6. Application Returned 404

The empty WAR was deployed to Tomcat.

Tomcat itself started successfully, but accessing:

<http://SERVER-IP:8084/SimpleCustomerApp/>

returned:

HTTP Status 404 – Not Found

This showed that Tomcat was working but the application artifact was incorrect.

7. Ant Build Tool

The repository already contained:

build.xml

Therefore Ant was used instead of Maven.

Ant was installed using:

```
dnf install -y ant
```

Initially Ant failed because:

JAVA_HOME is not set

We configured:

```
export JAVA_HOME=/usr/lib/jvm/java-21-amazon-corretto.x86_64
```

Verified using:

```
ant -version
```

Output:

```
Apache Ant(TM) version 1.10.12
```

8. Existing build.xml Problem

Running:

```
ant getwar
```

failed because the repository's existing `build.xml` referenced old tools such as:

```
FindBugs  
Cobertura  
PMD  
JUnit
```

and contained old Windows-related configuration.

Example error:

```
taskdef class edu.umd.cs.findbugs.anttask.FindBugsTask cannot be found
```

Therefore we created a separate Jenkins-compatible Ant build file:

```
build-jenkins.xml
```

9. Prepare Tomcat Libraries

The application required Tomcat libraries during compilation.

They were copied from the existing Tomcat container:

```
mkdir -p .tomcat-lib
```

```
docker cp \
simplecustomerapp-tomcat:/usr/local/tomcat/lib/. \
.tomcat-lib/
```

This provided libraries such as:

```
servlet-api.jar
jsp-api.jar
catalina.jar
```

10. Build Application Using Ant

The Jenkins-compatible Ant build command was:

```
ant -f build-jenkins.xml getwar
```

Successful output:

```
[war] Building war:
/var/lib/jenkins/workspace/Task-02-SimpleCustomerApp-Pipeline/build/war/SimpleCustomerApp.
war
```

BUILD SUCCESSFUL

The correct WAR was created at:

```
build/war/SimpleCustomerApp.war
```

This WAR contained the real web application files.

11. Upload Artifact to Nexus

Jenkins used stored Nexus credentials:

```
nexus-credentials
```

WAR file:

```
build/war/SimpleCustomerApp.war
```

Upload command:

```
curl --fail \  
-u "NEXUS_USER:NEXUS_PASS" \  
--upload-file build/war/SimpleCustomerApp.war \  
"NEXUS_URL/repository/NEXUS_REPO/"
```

Purpose of Nexus

Nexus stores application artifacts centrally.

Instead of keeping the WAR only inside the Jenkins workspace, it is stored in an artifact repository where it can later be downloaded or deployed.

12. Tomcat Docker Container

An existing Tomcat container from Task 1 was already using:

8083

Container:

tomcat-hiring

Therefore Task 2 used a separate Tomcat container:

simplecustomerapp-tomcat

and port:

8084

Container creation:

```
docker run -d \  
--name simplecustomerapp-tomcat \  
-p 8084:8080 \  
tomcat:9.0-jdk17-temurin
```

Meaning:

Host Port → 8084

Container Port → 8080

13. Deploy WAR to Tomcat

Old application files were removed:

```
docker exec simplecustomerapp-tomcat \  
rm -rf \  
/usr/local/tomcat/webapps/SimpleCustomerApp \  
/usr/local/tomcat/webapps/SimpleCustomerApp.war
```

The new WAR was copied:

```
docker cp \  
build/war/SimpleCustomerApp.war \  
simplecustomerapp-tomcat:/usr/local/tomcat/webapps/SimpleCustomerApp.war
```

Tomcat was restarted:

```
docker restart simplecustomerapp-tomcat
```

Tomcat automatically extracted:

SimpleCustomerApp.war

into:

```
/usr/local/tomcat/webapps/SimpleCustomerApp/
```

14. Application Health Check

The Jenkins pipeline checks whether the application is actually responding.

```
curl http://localhost:8084/SimpleCustomerApp/
```

The pipeline repeatedly checks the HTTP response code.

Expected:

200

If HTTP status is 200:

SUCCESS: SimpleCustomerApp deployed.

If it does not become healthy:

docker logs --tail 100 simplecustomerapp-tomcat

is executed for troubleshooting.

15. Application URL

Final application URL:

<http://44.223.94.171:8084/SimpleCustomerApp/>

The application successfully opened in the browser.

16. Slack Notification

Slack was configured using:

#jenkins-alerts

On success:

SUCCESS: Task-02-SimpleCustomerApp-Pipeline
SimpleCustomerApp deployed successfully on port 8084

On failure:

FAILED: Task-02-SimpleCustomerApp-Pipeline

This provides real-time pipeline notifications.

17. Jenkins Permission Issue

After manually running commands as **root**, some files inside the Jenkins workspace became owned by root.

Jenkins then failed during Git checkout:

Permission denied
build/classes/Sample.txt
build/war/Sample.txt

Why?

Manual build executed as root



Files owned by root



Jenkins runs as jenkins user



Jenkins cannot overwrite files



Git checkout fails

Fixed using:

```
chown -R jenkins:jenkins \  
/var/lib/jenkins/workspace/Task-02-SimpleCustomerApp-Pipeline
```

After correcting ownership, Jenkins could access the workspace normally.

18. Final Successful Result

The complete pipeline finally executed successfully:

Git Clone



SonarQube Analysis



Prepare Tomcat Libraries



Ant Build



Upload Artifact to Nexus



Deploy to Tomcat



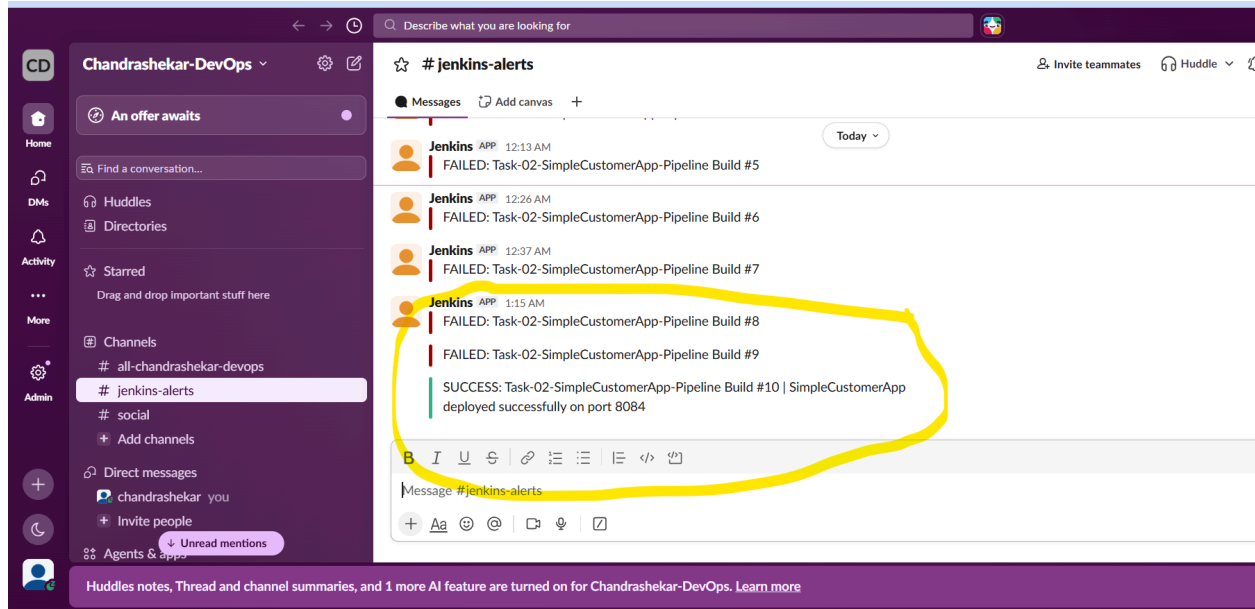
Application Health Check



Slack Notification

The screenshot shows the Jenkins web interface. The top navigation bar includes the Jenkins logo and the pipeline name 'Task-02-SimpleCustomerApp-Pipeline'. The left sidebar contains a list of actions: Status, Changes, Build Now, Delete Pipeline, Configure, Rename, SonarQube, Stages, and Pipeline Syntax. The main content area displays the pipeline's status as 'Success' with a green checkmark. Below this, the 'SonarQube Quality Gate' section shows 'SimpleCustomerApp' as 'Passed' and 'server-side processing' as 'Success'. A 'Permalinks' section lists the last build, stable build, successful build, and completed build, all from 1 minute and 4 seconds ago. A 'Builds' section on the left shows a list of builds with a filter input and a dropdown menu. The bottom of the interface shows the REST API and a 'Relaunch to update' button.

The screenshot shows a web form titled 'Your basic info' with a blue header. The form contains five input fields: 'Full Name:', 'Email:', 'Password:', 'Mobile:', and 'Address:'. Each field has a corresponding text input box. The form is set against a light blue background. At the top of the page, there are three navigation links: 'HOME', 'APPLY NOW', and 'LEARNING'. The bottom of the page shows a system tray with a temperature indicator of 26°C, a taskbar with various application icons, and a system clock showing 01:22.



3) Setup a Jenkins CICD pipeline using Scripted pipeline using feature-1.1 branch.

Source Code:

https://github.com/betawins/sabear_simplecutomerapp/tree/feature-1.1

stages:

- Git Clone.
- SonarQube Integration.
- Maven Compilation.
- Nexus Artifactory.
- Slack Notification.
- Deploy On tomcat.

1. Objective

The purpose of this task was to automate the complete CI/CD process using a **Jenkins Scripted Pipeline**.

The pipeline performs:

Source Code Checkout



Code Quality Analysis
↓
Application Build
↓
Artifact Storage
↓
Application Deployment
↓
Application Verification
↓
Slack Notification

2. Tools Used

Tool	Purpose
Jenkins	CI/CD automation
GitHub	Source code repository
SonarQube	Code quality analysis
Maven	Application compilation/build
Nexus	Artifact repository
Docker	Container platform
Tomcat	Application server
Slack	Pipeline notification

3. Jenkins Job Creation

Go to:

Jenkins Dashboard
→ New Item

Enter job name:

Task-03-SimpleCustomerApp-Scripted

Select:

Pipeline

Click:

OK

Under the Pipeline section select:

Definition → Pipeline script

4. Scripted Pipeline Structure

Task 3 specifically required a **Scripted Pipeline**.

Basic syntax:

```
node {  
  
    stage('Git Clone') {  
        // commands  
    }  
  
    stage('SonarQube Integration') {  
        // commands  
    }  
  
    stage('Maven Compilation') {  
        // commands  
    }  
  
    stage('Nexus Artifactory') {  
        // commands  
    }  
  
    stage('Deploy On Tomcat') {  
        // commands  
    }  
}
```

Unlike Declarative Pipeline, Scripted Pipeline does not use:

```
pipeline {  
  agent any  
  
  stages {  
    stage() {  
      steps {  
      }  
    }  
  }  
}
```

Instead, Scripted Pipeline mainly uses:

```
node {  
  stage() {  
  }  
}
```

5. Create Separate Tomcat Container

Task 2 was already using Tomcat on another port, so Task 3 used a separate Tomcat instance.

First check running containers:

```
docker ps
```

Check whether port **8085** is free:

```
ss -ltnp | grep 8085
```

No output means port **8085** is available.

Create Tomcat container:

```
docker run -d \  
  --name simplecustomerapp-scripted \  
  -p 8085:8080 \  
  tomcat:9.0-jdk17-temurin
```

Verify:

`docker ps`

Result:

`simplecustomerapp-scripted`
`8085 → 8080`

Meaning:

EC2 Host Port = 8085
Tomcat Container Port = 8080

6. Stage 1 — Git Clone

The first stage clones the `feature-1.1` branch.

```
stage('Git Clone') {  
  
    echo "Cloning feature-1.1 branch..."  
  
    git branch: 'feature-1.1',  
        url: 'https://github.com/betawins/sabear_simplecutomerapp.git'  
}
```

Purpose

Jenkins downloads the latest application source code from GitHub into its workspace.

Repository:

`sabear_simplecutomerapp`

Branch:

`feature-1.1`

7. Stage 2 — SonarQube Integration

SonarQube performs static code analysis.

```
stage('SonarQube Integration') {
```

```
def scannerHome = tool 'SonarScanner'

withSonarQubeEnv('SonarQube-Server') {

    sh """
        ${scannerHome}/bin/sonar-scanner \
        -Dsonar.projectKey=SimpleCustomerApp-Scripted \
        -Dsonar.projectName=SimpleCustomerApp-Scripted \
        -Dsonar.sources=src
    """
}
}
```

Purpose

SonarQube checks code for:

- Bugs
- Vulnerabilities
- Code smells
- Duplications
- Maintainability issues

Project key used:

SimpleCustomerApp-Scripted

8. Stage 3 — Maven Compilation

The project used an older application structure:

```
src/
WebContent/
```

Standard Maven normally expects:

```
src/main/java/
src/main/webapp/
```

Because of this, plain:

```
mvn clean package
```

previously created an almost empty WAR.

For Task 3, Maven was still required, so a Jenkins-specific Maven configuration was generated:

pom-task3.xml

This configuration tells Maven how to use:

src
WebContent
Tomcat libraries

9. Prepare Tomcat Libraries

Some Java classes require Servlet/Tomcat libraries during compilation.

The libraries were copied from the running Tomcat container:

```
rm -rf .tomcat-lib
```

```
mkdir -p .tomcat-lib
```

```
docker cp \
simplecustomerapp-scripted:/usr/local/tomcat/lib/ \
.tomcat-lib/
```

Examples include:

servlet-api.jar
jsp-api.jar
catalina.jar

These libraries are used only during compilation.

10. Maven Build

The pipeline runs:

```
mvn -f pom-task3.xml clean package
```

The custom Maven configuration:

1. Compiles Java files from:

src/

2. Uses web files from:

WebContent/

3. Uses Tomcat libraries from:

.tomcat-lib/

4. Generates:

target/SimpleCustomerApp.war

The WAR was verified using:

```
ls -lh target/SimpleCustomerApp.war
```

and:

```
jar tf target/SimpleCustomerApp.war
```

11. Stage 4 — Nexus Artifactory

After successful Maven compilation, Jenkins uploads the WAR to Nexus.

Credentials were stored securely in Jenkins:

nexus-credentials

Pipeline section:

```
withCredentials([
    usernamePassword(
        credentialsId: 'nexus-credentials',
        usernameVariable: 'NEXUS_USER',
        passwordVariable: 'NEXUS_PASS'
    )
]) {
```

```

sh """
  curl --fail \
  -u "${NEXUS_USER}:${NEXUS_PASS}" \
  --upload-file target/SimpleCustomerApp.war \
  "${NEXUS_URL}/repository/
{NEXUS_REPO}/SimpleCustomerApp-Scripted-${BUILD_NUMBER}.war"
"""
}

```

Purpose of Nexus

Nexus centrally stores the generated application artifact.

Instead of keeping the WAR only inside Jenkins:

```

Jenkins Build
  ↓
WAR
  ↓
Nexus Repository

```

Each Jenkins build can have its own artifact version.

Example:

```

SimpleCustomerApp-Scripted-1.war
SimpleCustomerApp-Scripted-2.war

```

12. Stage 5 — Deploy On Tomcat

Container used:

simplecustomerapp-scripted

Port:

8085

Before deployment, the previous application is removed:

```

docker exec simplecustomerapp-scripted \
rm -rf \
/usr/local/tomcat/webapps/SimpleCustomerApp \

```

/usr/local/tomcat/webapps/SimpleCustomerApp.war

Then the newly generated WAR is copied:

```
docker cp \
target/SimpleCustomerApp.war \
simplecustomerapp-scripted:/usr/local/tomcat/webapps/SimpleCustomerApp.war
```

Restart Tomcat:

```
docker restart simplecustomerapp-scripted
```

Tomcat automatically deploys:

SimpleCustomerApp.war

13. Application Health Check

Deployment success does not always mean the application is working.

Therefore the pipeline checks:

<http://localhost:8085/SimpleCustomerApp/>

Using:

curl

The pipeline waits and checks repeatedly.

Expected response:

HTTP 200

When HTTP status is 200:

SUCCESS: Application is running.

If the application does not become healthy, Jenkins checks Tomcat logs:

```
docker logs --tail 100 simplecustomerapp-scripted
```

and fails the pipeline.

14. Stage 6 — Slack Notification

If all stages are successful:

```
slackSend(  
  channel: '#jenkins-alerts',  
  color: 'good',  
  message: "SUCCESS: env.JOB_NAMEBuild#{env.BUILD_NUMBER}"  
)
```

Slack receives a success message.

If any stage fails, the `catch` block executes:

```
catch (err) {  
  
  slackSend(  
    channel: '#jenkins-alerts',  
    color: 'danger',  
    message: "FAILED: env.JOB_NAMEBuild#{env.BUILD_NUMBER}"  
  )  
  
  throw err  
}
```

This sends a failure notification.

15. Why We Used try/catch/finally

In Scripted Pipeline we used:

```
try {  
  // Pipeline stages  
}  
catch (err) {  
  // Failure handling  
}  
finally {  
  // Always executed  
}
```

try

Runs the main pipeline.

catch

Runs when any pipeline stage fails.

Used for:

Slack failure notification

Error handling

finally

Runs whether the pipeline succeeds or fails.

Example:

```
finally {  
    echo "Task 3 Scripted Pipeline execution completed."  
}
```

16. Final Application URL

The application was deployed successfully at:

<http://<EC2-PUBLIC-IP>:8085/SimpleCustomerApp/>

Task 3 uses:

8085

17. Final Pipeline Flow

GitHub

↓

feature-1.1

↓

Jenkins Scripted Pipeline

↓
Git Clone
↓
SonarQube Analysis
↓
Prepare Tomcat Libraries
↓
Maven Compilation
↓
SimpleCustomerApp.war
↓
Nexus Artifactory
↓
Tomcat Deployment
↓
Port 8085
↓
Application Health Check
↓
Slack Notification

18. Final Result

Jenkins console output showed:

Slack Notification

Task 3 Scripted Pipeline execution completed.

Finished: SUCCESS

Therefore Task 3 was successfully completed.

19. Declarative vs Scripted Pipeline

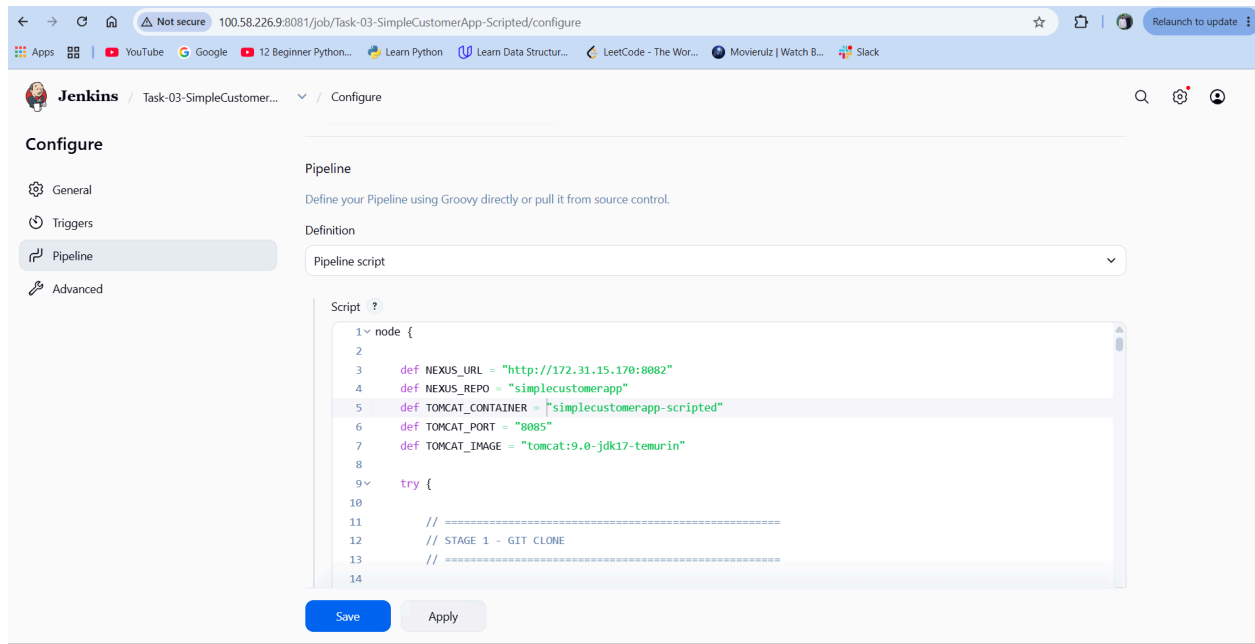
Declarative Pipeline

Uses `pipeline {}`

Scripted Pipeline

Uses `node {}`

Uses agent	Node decides execution
Uses stages <code>{}</code>	Direct stage() blocks
Uses steps <code>{}</code>	Commands directly inside stage
Easier to read	More flexible
Structured syntax	Groovy-based programming
Good for standard CI/CD	Good for complex logic



← → ↻ 🏠 **Not secure** 3.91.15.127:9000/dashboard?id=SimpleCustomerApp-Scripted&codeScope=overall 🔍 ☆ 📄 🔄 🧑 Relaunch to update

📱 Apps 📺 YouTube 🌐 Google 📺 12 Beginner Python... 📺 Learn Python 📺 Learn Data Structur... 📺 LeetCode - The Wor... 📺 Movierulz | Watch B... 📺 Slack

🔔 Keep your instance current and get the **latest SonarQube Community Build**, available now. ✕

🔔 There's a new version of SonarQube Server available. Upgrade to SonarQube Server and get access to enterprise features. [Learn More](#) ✕

🏠 **SonarQube** community Projects Issues Rules Quality profiles Quality gates Administration More 🔍 🔔 ⓘ 📄 A

📄 **SimpleCustomerApp-Scripted** Project

Analysis

Overview

Issues

🔒 Security h... **Deprecated**

Reporting

📊 Measures

📈 Activity

📈 Quality gate history

Policies

📄 Quality profiles

🔒 Quality gate

Project

📄 Code

SimpleCustomerApp-Scripted / Overview

Overview [Set as homepage](#) ☆

207 Lines of Code • Version 2.0 • Last analysis 7 minutes ago

Quality gate ⓘ **Passed** ⚡

⚠ The last analysis has warnings. [See details](#)

New Code **Overall Code**

Security 1 Open issues B	Reliability 0 Open issues A	Maintainability 2 Open issues A
---	--	--

← → ↻ 🏠 **Not secure** 100.58.226.9:8081/job/Task-03-SimpleCustomerApp-Scripted/ ☆

📱 Apps 📺 YouTube 🌐 Google 📺 12 Beginner Python... 📺 Learn Python 📺 Learn Data Structur... 📺 LeetCode - The Wor... 📺 Movierulz | Watch B... 📺 Slack

Jenkins / Task-03-SimpleCustomerApp-Scripted

Status

📄 Changes

▶ Build Now

🗑 Delete Pipeline

⚙ Configure

✎ Rename

📄 SonarQube

📄 Stages

🔍 Pipeline Syntax

✅ **Task-03-SimpleCustomerApp-Scripted**

SonarQube Quality Gate

SimpleCustomerApp-Scripted **Passed**

server-side processing: **Success**

Permalinks

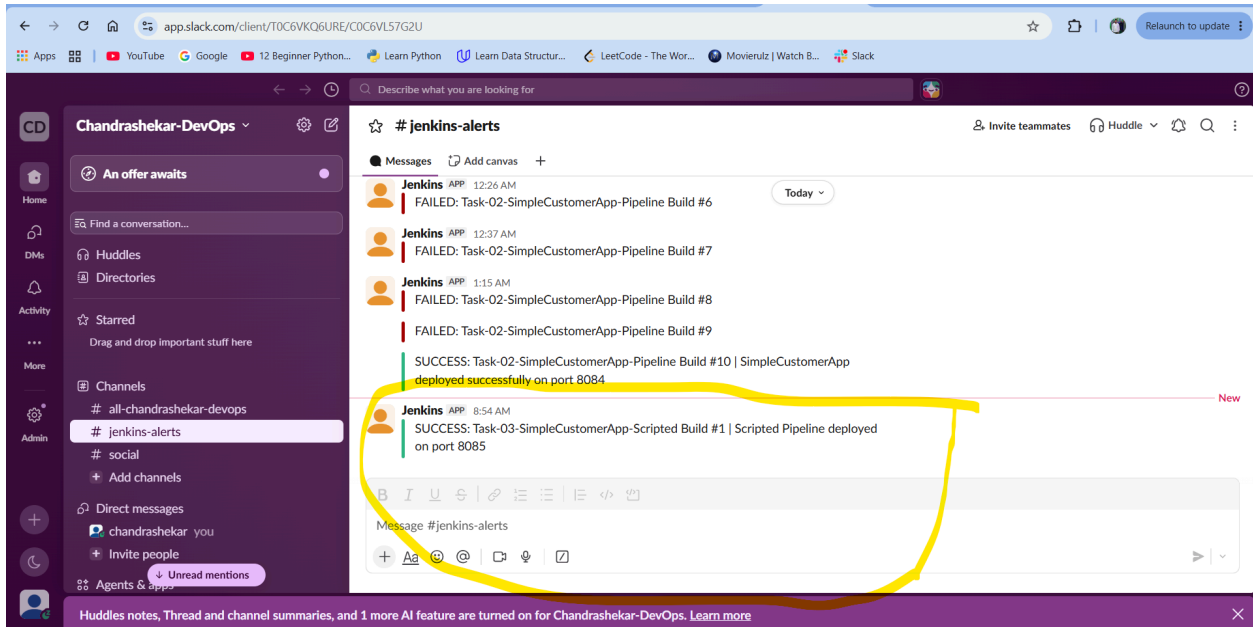
- [Last build \(#1\), 2 min 31 sec ago](#)
- [Last stable build \(#1\), 2 min 31 sec ago](#)
- [Last successful build \(#1\), 2 min 31 sec ago](#)
- [Last completed build \(#1\), 2 min 31 sec ago](#)

Builds > ⋮

🔍 Filter /

Today

✅ #1 3:24 AM 📄 ⌵



“I created a Jenkins Scripted CI/CD pipeline for a Java web application using the feature-1.1 branch. Jenkins clones the source code from GitHub, performs code quality analysis using SonarQube, compiles and packages the application using Maven, uploads the generated WAR artifact to Nexus, deploys the WAR into a Docker-based Tomcat server, performs an HTTP health check, and sends the final build status to Slack. I used `node`, `stage`, and `try-catch-finally` because this task required a Scripted Pipeline.

4) Write sample skeleton of pipelines.

Task 4 — Write Sample Skeleton of Jenkins Pipelines

The purpose is to understand the basic structure of both **Declarative Pipeline** and **Scripted Pipeline**.

You do not need SonarQube, Nexus, Tomcat, or Docker for this task unless your trainer specifically asks you to execute a real application.

1. Declarative Pipeline Skeleton

Declarative Pipeline uses a structured syntax.

```
pipeline {
```

agent any

```
stages {  
  
    stage('Git Clone') {  
        steps {  
            echo 'Cloning source code...'  
        }  
    }  
  
    stage('Build') {  
        steps {  
            echo 'Building application...'  
        }  
    }  
  
    stage('Test') {  
        steps {  
            echo 'Testing application...'  
        }  
    }  
  
    stage('Deploy') {  
        steps {  
            echo 'Deploying application...'  
        }  
    }  
}  
  
post {  
  
    success {  
        echo 'Pipeline completed successfully.'  
    }  
  
    failure {  
        echo 'Pipeline failed.'  
    }  
}
```

Basic structure

pipeline

↓
agent
↓
stages
↓
stage
↓
steps
↓
post

The important keywords are:

pipeline
agent
stages
stage
steps
post

2. Scripted Pipeline Skeleton

Scripted Pipeline uses Groovy-style syntax and normally starts with `node`.

```
node {  
  
    try {  
  
        stage('Git Clone') {  
  
            echo 'Cloning source code...'  
        }  
  
        stage('Build') {  
  
            echo 'Building application...'  
        }  
  
        stage('Test') {  
  
            echo 'Testing application...'  
        }  
    }  
}
```

```

    stage('Deploy') {
        echo 'Deploying application...'
    }

    echo 'Pipeline completed successfully.'

}

catch (err) {

    echo "Pipeline failed: ${err}"

    throw err
}

finally {

    echo 'Pipeline execution completed.'
}
}

```

Basic structure:

```

node
↓
try
↓
stage
↓
commands
↓
catch
↓
finally

```

3. Simple Real-Time Declarative Example

```

pipeline {
    agent any

```

```
stages {  
  
    stage('Checkout') {  
        steps {  
            git 'https://github.com/example/application.git'  
        }  
    }  
  
    stage('Build') {  
        steps {  
            sh 'mvn clean package'  
        }  
    }  
  
    stage('Test') {  
        steps {  
            sh 'mvn test'  
        }  
    }  
  
    stage('Deploy') {  
        steps {  
            echo 'Deploying application...'  
        }  
    }  
}  
  
post {  
  
    success {  
        echo 'Build Successful'  
    }  
  
    failure {  
        echo 'Build Failed'  
    }  
}  
}
```

4. Simple Real-Time Scripted Example

```
node {  
  try {  
    stage('Checkout') {  
      git 'https://github.com/example/application.git'  
    }  
  
    stage('Build') {  
      sh 'mvn clean package'  
    }  
  
    stage('Test') {  
      sh 'mvn test'  
    }  
  
    stage('Deploy') {  
      echo 'Deploying application...'  
    }  
  
    echo 'Build Successful'  
  }  
  
  catch (err) {  
    echo 'Build Failed'  
  
    throw err  
  }  
}
```

5. Declarative vs Scripted

Declarative

Starts with `pipeline {}`

Scripted

Starts with `node {}`

Uses agent	Uses node/executor
Uses stages {}	Direct stage()
Uses steps {}	Commands directly inside stage
Easier to learn	More flexible
Strict structure	Groovy programming style
Good for standard pipelines	Good for complex pipeline logic

You already used both practically:

Task 2 → Declarative Pipeline

Task 3 → Scripted Pipeline

So Task 4 is mainly showing that you understand their **skeleton/structure**.

6. How to Do Task 4 in Jenkins

Create:

Dashboard
 → New Item
 → Task-04-Pipeline-Skeleton
 → Pipeline
 → OK

For the first test, paste:

```
pipeline {
    agent any

    stages {
        stage('Stage 1') {
            steps {
                echo 'This is Stage 1'
            }
        }

        stage('Stage 2') {
```

```

        steps {
            echo 'This is Stage 2'
        }
    }

    stage('Stage 3') {
        steps {
            echo 'This is Stage 3'
        }
    }
}

```

Click:

Save

→ Build Now

Expected:

Stage 1

Stage 2

Stage 3

Finished: SUCCESS

Then you can replace it with the Scripted skeleton:

```

node {

    stage('Stage 1') {
        echo 'This is Stage 1'
    }

    stage('Stage 2') {
        echo 'This is Stage 2'
    }

    stage('Stage 3') {
        echo 'This is Stage 3'
    }
}

```

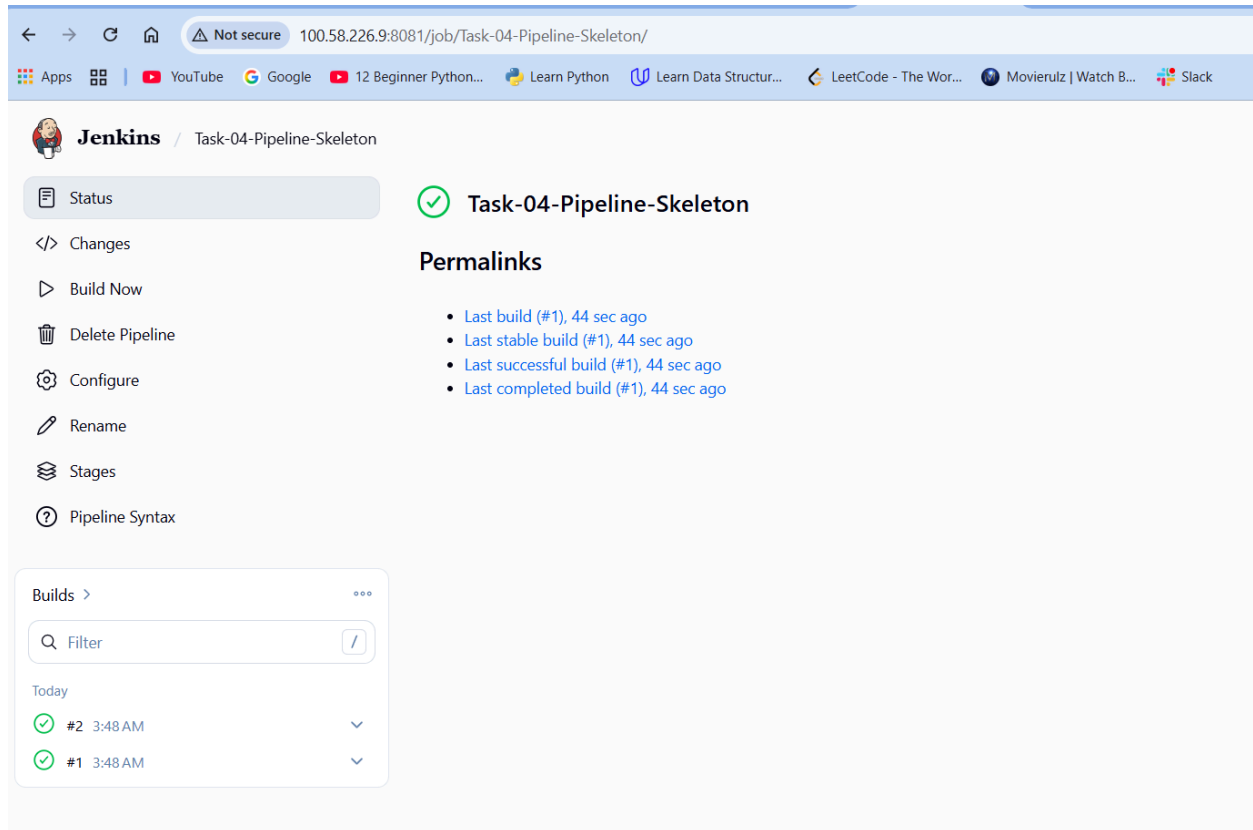
Again:

Save

→ Build Now

Expected:

Finished: SUCCESS



The screenshot shows the Jenkins web interface for a job named 'Task-04-Pipeline-Skeleton'. The browser address bar indicates the URL '100.58.226.9:8081/job/Task-04-Pipeline-Skeleton/'. The Jenkins logo and job name are at the top. On the left, a sidebar contains links: Status (selected), Changes, Build Now, Delete Pipeline, Configure, Rename, Stages, and Pipeline Syntax. The main area shows a green checkmark icon and the job name 'Task-04-Pipeline-Skeleton'. Below this, a 'Permalinks' section lists four links: 'Last build (#1), 44 sec ago', 'Last stable build (#1), 44 sec ago', 'Last successful build (#1), 44 sec ago', and 'Last completed build (#1), 44 sec ago'. At the bottom, a 'Builds' section shows a list of builds for 'Today': build #2 at 3:48 AM and build #1 at 3:48 AM, both with green checkmark icons.

Task 4: Sample Pipeline Skeletons

In this task, I created sample Jenkins pipeline skeletons to understand the basic syntax and structure of Declarative and Scripted Pipelines. Declarative Pipeline uses **pipeline**, **agent**, **stages**, **stage**, and **steps**, whereas Scripted Pipeline uses **node** and direct **stage** blocks. Declarative Pipeline provides a structured and easy-to-read format, while Scripted Pipeline provides more flexibility using Groovy programming logic.

5) Create a parameterized job in Jenkins.

Source Code:

<https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git>

Jenkins-04 — Task 5 Documentation

Create a Parameterized Job in Jenkins

Task Requirement

Create a **Parameterized Jenkins Job** using the source code:

<https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git>

The job should allow the user to choose values before running the build.

1. Objective

The purpose of this task is to understand how Jenkins parameters work.

Instead of hard-coding values inside the pipeline, we allow the user to select values such as:

Branch Name

Environment

Whether Build Should Run

Normal Jenkins job:

Build Now



Fixed values

Parameterized Jenkins job:

Build with Parameters



User selects values

↓
Pipeline uses selected values

2. Jenkins Job Creation

Go to:

Jenkins Dashboard
→ New Item

Enter job name:

Task-05-Parameterized-Job

Select:

Pipeline

Click:

OK

3. Enable Parameterized Job

Go to:

Job
→ Configure
→ General

Enable:

This project is parameterized

We created three parameters.

Parameter	Type	Value
BRANCH_NAME	String Parameter	master

ENVIRONME Choice Parameter DEV / TEST / PROD
NT

RUN_BUILD Boolean true
Parameter

4. String Parameter

Select:

Add Parameter
→ String Parameter

Configure:

Name:
BRANCH_NAME

Default Value:
master

Description:
Enter Git branch name

Purpose

This parameter decides which Git branch Jenkins should clone.

Example:

master
develop
feature

Pipeline usage:

params.BRANCH_NAME

5. Choice Parameter

Select:

Add Parameter
→ Choice Parameter

Configure:

Name:
ENVIRONMENT

Choices:

DEV
TEST
PROD

Description:

Select deployment environment

Purpose

This allows the user to select the required environment.

Example:

DEV
TEST
PROD

Pipeline usage:

params.ENVIRONMENT

6. Boolean Parameter

Select:

Add Parameter
→ Boolean Parameter

Configure:

Name:

RUN_BUILD

Default:

Checked / true

Description:

Run Maven build?

Purpose

This parameter decides whether Jenkins should execute the Maven Build stage.

Pipeline usage:

params.RUN_BUILD

7. Pipeline Structure

The pipeline contained these stages:

Display Parameters



Git Clone



Maven Build



Environment Selection



Post Build Result

8. Display Parameters Stage

This stage displays the values selected by the user.

```
stage('Display Parameters') {
```

```
    steps {
```

```
        echo "Branch Name : ${params.BRANCH_NAME}"
```

```
        echo "Environment : ${params.ENVIRONMENT}"
```

```
        echo "Run Build  : ${params.RUN_BUILD}"
    }
}
```

Example output:

Branch Name : master
Environment : DEV
Run Build : true

9. Git Clone Stage

The Git branch is selected dynamically using the `BRANCH_NAME` parameter.

```
stage('Git Clone') {
    steps {
        echo "Cloning branch: params.BRANCH_NAME" gitbranch: "${params.BRANCH_NAME}",
        url: 'https://github.com/betawins/spring3-mvc-maven-xml-hello-world-1.git'
    }
}
```

Purpose

If the user enters:

master

Jenkins clones the `master` branch.

If the user changes it to another valid branch, Jenkins can clone that branch instead.

10. Maven Build Stage

The Maven build should run only when:

`RUN_BUILD = true`

We used:

when {

expression {

return params.RUN_BUILD

}

}

This means:

RUN_BUILD = true

↓

Maven Build Runs

RUN_BUILD = false

↓

Maven Build Skipped

11. Java Compatibility Issue

During the first Maven build, we received:

Source option 6 is no longer supported.

Target option 6 is no longer supported.

We checked the project's `pom.xml` and found:

```
<jdk.version>1.6</jdk.version>
```

The Maven compiler used:

```
<source>jdk.version < /source >< target >{jdk.version}</target>
```

The project was therefore trying to compile using Java 6.

Our Jenkins server was using a modern Java version, so Java 6 compilation was rejected.

Solution

We passed:

-Djdk.version=1.8

to Maven.

This changed the build compatibility to Java 8.

12. Old Maven WAR Plugin Issue

After fixing Java, another error occurred:

```
maven-war-plugin:2.2
Cannot access defaults field of Properties
```

The old project was using Maven WAR Plugin [2.2](#).

That version is too old for the modern Java/Maven environment.

Solution

Instead of using:

```
mvn clean package
```

we explicitly used Maven WAR Plugin version:

```
3.4.0
```

Final command:

```
mvn compile \
  org.apache.maven.plugins:maven-war-plugin:3.4.0:war \
  -Djdk.version=1.8
```

13. Final Maven Build Stage

The working Maven stage was:

```
stage('Maven Build') {
    when {
```

```
expression {  
    return params.RUN_BUILD  
}  
}  
  
steps {  
  
    echo "Building application with Java 8 compatibility..."  
  
    sh ""  
        rm -rf target  
  
        mvn compile \  
            org.apache.maven.plugins:maven-war-plugin:3.4.0:war \  
            -Djdk.version=1.8  
  
        echo "Generated WAR:"  
  
        ls -lh target/*.war  
    ""  
}  
}
```

14. Generated WAR

The final build successfully generated:

target/ncodeit-hello-world-3.0.war

WAR size:

4.0 MB

This confirmed that the Maven application was compiled and packaged correctly.

15. Environment Selection Stage

The selected **ENVIRONMENT** parameter was checked using conditional logic.

```
stage('Environment Selection') {  
  
    steps {  
  
        echo "Selected Environment: ${params.ENVIRONMENT}"  
  
        script {  
  
            if (params.ENVIRONMENT == 'DEV') {  
  
                echo "Deploying to DEV environment..."  
  
            }  
            else if (params.ENVIRONMENT == 'TEST') {  
  
                echo "Deploying to TEST environment..."  
  
            }  
            else if (params.ENVIRONMENT == 'PROD') {  
  
                echo "Deploying to PROD environment..."  
  
            }  
        }  
    }  
}
```

For our successful build:

ENVIRONMENT = DEV

Output:

Selected Environment: DEV
Deploying to DEV environment...

16. Post Build Actions

If the job succeeds:

```
post {
```

```
success {  
  
    echo "Parameterized Job Completed Successfully."  
}  
  
failure {  
  
    echo "Parameterized Job Failed."  
}  
}
```

17. Parameters Used During Final Build

We selected:

BRANCH_NAME = master

ENVIRONMENT = DEV

RUN_BUILD = true

Then clicked:

Build

18. Final Successful Output

The Maven build showed:

BUILD SUCCESS

WAR generated:

target/ncodeit-hello-world-3.0.war

Environment:

Selected Environment: DEV

Deploying to DEV environment...

Final Jenkins result:

Parameterized Job Completed Successfully.

Finished: SUCCESS

19. Problems Faced and Solutions

Problem	Reason	Solution
Java 6 compilation error	Old project used Java 1.6	Used <code>-Djdk.version=1.8</code>
WAR plugin error	<code>maven-war-plugin:2.2</code> was too old	Used <code>maven-war-plugin:3.4.0</code>
Target directory permission error	Some files were created by root	Changed ownership to <code>jenkins:jenkins</code>
Maven clean failed	Jenkins could not delete root-owned files	Removed target and fixed permissions

Important ownership fix:

```
chown -R jenkins:jenkins \  
/var/lib/jenkins/workspace/Task-05-Parameterized-Job
```

20. Final Pipeline Flow

```
User clicks  
Build with Parameters  
↓  
Select BRANCH_NAME  
↓  
Select ENVIRONMENT  
↓  
Select RUN_BUILD  
↓  
Display Parameters  
↓  
Git Clone
```

↓
Maven Build
↓
Generate WAR
↓
Environment Selection
↓
Finished: SUCCESS

21. Why Parameterized Jobs Are Useful

Parameterized jobs make one Jenkins job reusable.

For example, instead of creating:

DEV Job
TEST Job
PROD Job

we can create one job:

Parameterized Jenkins Job

and choose:

DEV / TEST / PROD

when starting the build.

This reduces duplicate Jenkins jobs and makes CI/CD pipelines more flexible.

“I created a parameterized Jenkins pipeline where users can select the Git branch, deployment environment, and whether the Maven build should run. I used String, Choice, and Boolean parameters. The selected values are accessed using `params.PARAMETER_NAME`. The Git stage dynamically checks out the selected branch, the Maven stage runs based on the Boolean parameter, and conditional logic is used for DEV, TEST, or PROD environment selection. I also resolved Java and Maven plugin compatibility issues in the old application.”

What is a parameterized job?

“A parameterized job allows users to provide input values at build time. Those values can control branch selection, environment, version, deployment options, or whether a particular stage should execute.”

The image displays two screenshots of the Jenkins web interface, illustrating a parameterized job configuration and its execution.

Top Screenshot: Pipeline Task-05-Parameterized-Job Configuration

The browser address bar shows the URL: `100.58.226.9:8081/job/Task-05-Parameterized-Job/build?delay=0sec`. The Jenkins interface shows the configuration page for the job "Task-05-Parameterized-Job".

Left Sidebar (Navigation):

- Status
- Changes
- Build with Parameters
- Delete Pipeline
- Configure
- Rename
- Stages
- Pipeline Syntax

Main Content Area:

Pipeline Task-05-Parameterized-Job

This build requires parameters:

BRANCH_NAME
Enter Git branch name
Input field: `master`

ENVIRONMENT
Select deployment environment
Dropdown menu: `DEV`

☒ **RUN_BUILD**
Run Maven build?

Builds:

- Today
- #3 4:04 AM
- #2 4:02 AM
- #1 3:58 AM

Buttons: `Build` (green), `Cancel` (grey)

Bottom Screenshot: Task-05-Parameterized-Job Build Status

The browser address bar shows the URL: `100.58.226.9:8081/job/Task-05-Parameterized-Job/`. The Jenkins interface shows the build status page for the job "Task-05-Parameterized-Job".

Left Sidebar (Navigation):

- Status
- Changes
- Build with Parameters
- Delete Pipeline
- Configure
- Rename
- Stages
- Pipeline Syntax

Main Content Area:

Task-05-Parameterized-Job

Permalinks

- Last build (#7), 51 sec ago
- Last stable build (#7), 51 sec ago
- Last successful build (#7), 51 sec ago
- Last completed build (#7), 51 sec ago

Builds:

- Today
- #7 4:29 AM

6) Setup one slave machine for Jenkins.

1. Objective

The purpose of this task is to understand Jenkins **Controller-Agent Architecture**.

The Jenkins controller manages:

- Jobs
- Pipelines
- Scheduling
- Plugins
- Credentials
- Build coordination

The Jenkins agent executes the actual build jobs.

Flow:

Jenkins Controller



SSH



Jenkins Agent



Execute Build

2. Existing Agent Machine

An existing EC2 instance was already available:

Jenkins-Agent-01

So a new EC2 instance was not required.

The agent was already configured in Jenkins and shown as:

Online

3. Verify Jenkins Node

Go to:

Jenkins Dashboard
→ Manage Jenkins
→ Nodes

Available nodes were:

Built-In Node
Jenkins-Agent-01
Jenkins-Agent-02

Jenkins-Agent-01 was online and available.

4. Agent Labels

Open:

Manage Jenkins
→ Nodes
→ Jenkins-Agent-01

The configured labels were:

agent01
linux

We used:

agent01

in the pipeline.

Why labels are used

Labels allow Jenkins to decide on which agent a job should run.

Example:

```
agent {
```

```
    label 'agent01'  
}
```

This tells Jenkins:

Run this pipeline only on the node having label agent01.

5. Jenkins Agent Workspace

The agent was configured with the remote workspace:

/home/jenkins/agent

When Jenkins runs a job, it creates a workspace such as:

/home/jenkins/agent/workspace/Task-06-Agent-Test

6. Create Test Pipeline Job

Go to:

Jenkins Dashboard
→ New Item

Job name:

Task-06-Agent-Test

Select:

Pipeline

Click:

OK

7. Agent Test Pipeline

The following pipeline was used:

```
pipeline {  
  
    agent {  
        label 'agent01'  
    }  
  
    stages {  
  
        stage('Agent Test') {  
  
            steps {  
  
                sh '''  
                    echo "Running on Jenkins Agent"  
                    hostname  
                    whoami  
                    pwd  
                    java -version  
                    git --version  
                '''  
            }  
        }  
  
        stage('Success') {  
  
            steps {  
  
                echo "Jenkins Agent setup successful!"  
            }  
        }  
    }  
}
```

8. Purpose of Test Commands

The pipeline uses the following commands:

hostname

Purpose:

Displays the hostname of the machine where the job is running.

Command:

whoami

Purpose:

Shows which Linux user is executing the Jenkins job.

Command:

pwd

Purpose:

Displays the current Jenkins workspace directory.

Command:

java -version

Purpose:

Checks whether Java is installed on the agent.

Command:

git --version

Purpose:

Checks whether Git is installed on the agent.

9. First Build Result

The first build started correctly on:

Jenkins-Agent-01

Workspace:

/home/jenkins/agent/workspace/Task-06-Agent-Test

The output showed:

whoami

jenkins

Java was available:

OpenJDK 21

But Git failed:

git: command not found

Because Git was not installed on the agent.

10. Install Git on Jenkins Agent

Connect to:

Jenkins-Agent-01

Install Git:

```
sudo dnf install git -y
```

Verify:

```
git --version
```

Output:

```
git version 2.50.1
```

11. Re-run Jenkins Job

After installing Git:

Task-06-Agent-Test

→ Build Now

Jenkins again scheduled the job on:

Jenkins-Agent-01

12. Successful Agent Output

The final console output confirmed:

Running on Jenkins-Agent-01

Workspace:

/home/jenkins/agent/workspace/Task-06-Agent-Test

Hostname:

ip-172-31-12-196.ec2.internal

User:

jenkins

Java:

openjdk version "21.0.12.1"

Git:

git version 2.50.1

Final message:

Jenkins Agent setup successful!

Final result:

Finished: SUCCESS

13. Final Architecture

Jenkins Controller

100.58.226.9:8081

|

| SSH / Jenkins Agent Connection

|

v

Jenkins-Agent-01

|

```
| Label: agent01
|
v
/home/jenkins/agent
|
v
Pipeline Build Execution
```

14. Why Jenkins Agents Are Used

Jenkins agents are useful because:

- Build workload is moved away from the controller
- Multiple jobs can run on different machines
- Different agents can have different tools
- Linux and Windows agents can be used separately
- Agents help scale Jenkins
- Heavy builds do not overload the controller
- Different environments can be isolated

Example:

Jenkins Controller

↓

Linux Agent → Java builds

Windows Agent → .NET builds

Docker Agent → Container builds

Kubernetes Agent → Cloud-native builds

15. Controller vs Agent

Jenkins Controller

Manages Jenkins

Stores configuration

Jenkins Agent

Executes builds

Has build tools

Schedules jobs	Runs commands
Manages credentials	Uses workspace
Runs Jenkins UI	Executes pipeline stages

16. Important Agent Configuration

For this task:

Node Name:
Jenkins-Agent-01

Label:
agent01

OS:
Linux

Workspace:
/home/jenkins/agent

User:
jenkins

Java:
21

Git:
2.50.1

17. Final Pipeline Flow

User clicks Build Now



Jenkins Controller reads pipeline



Find node with label agent01
↓
Schedule job on Jenkins-Agent-01
↓
Create workspace
↓
Run shell commands
↓
Verify Java
↓
Verify Git
↓
Build completes
↓
Finished: SUCCESS

18. Issues Faced and Solution

Issue	Reason	Solution
<code>git: command not found</code>	Git was not installed on agent	Installed Git using <code>dnf install git -y</code>
Success stage skipped initially	Agent Test stage failed because Git command failed	Installed Git and reran pipeline

19. Final Result

The test pipeline successfully ran on:

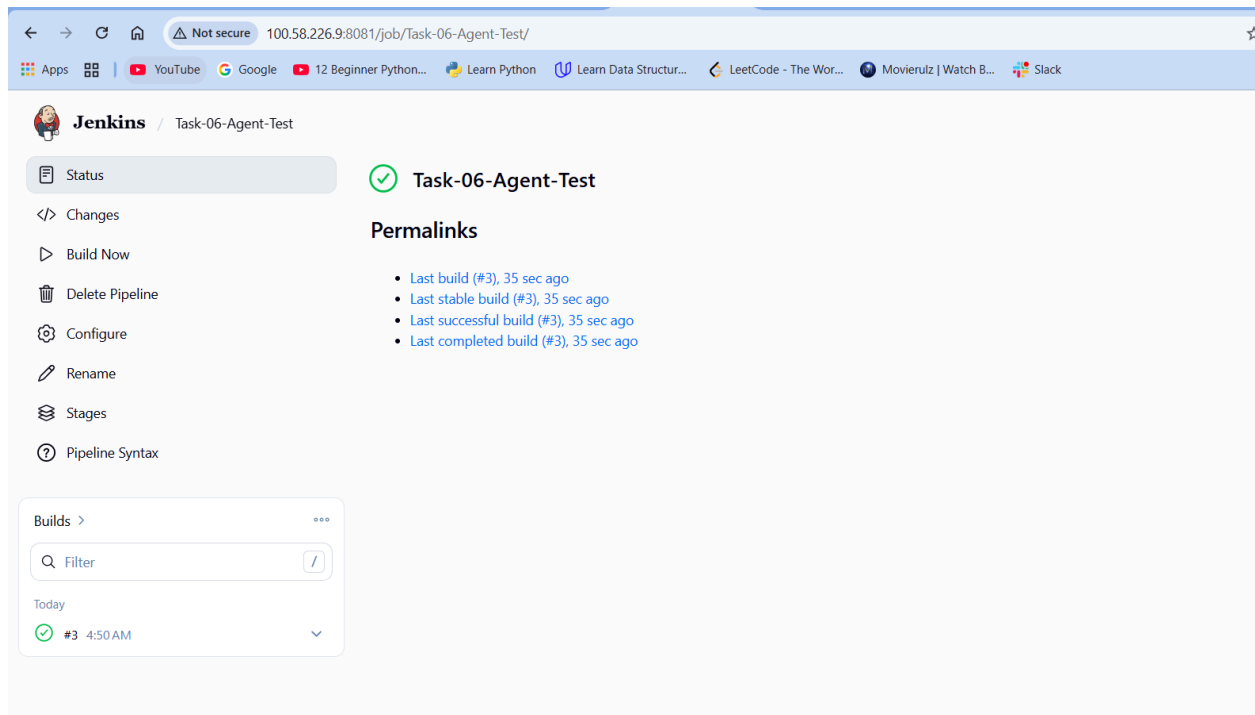
Jenkins-Agent-01

and ended with:

Jenkins Agent setup successful!

Finished: SUCCESS

Therefore the Jenkins agent setup was successfully verified.



“I configured and verified a Jenkins agent machine. The Jenkins controller manages and schedules jobs, while the agent executes the actual build. I used an existing Amazon Linux EC2 instance as Jenkins-Agent-01 with the label `agent01`. The agent workspace was `/home/jenkins/agent`. I created a test pipeline using `agent { label 'agent01' }` and verified hostname, user, Java and Git versions. Initially Git was missing, so I installed Git on the agent and reran the job successfully.”

“Agents help distribute build workload, improve scalability, isolate environments, and allow different machines to have different tools and operating systems.”

Jenkins Official Documentation Recommended

- Pipeline: <https://www.jenkins.io/doc/book/pipeline/>.
- Environment Variables:
<https://www.jenkins.io/doc/book/pipeline/jenkinsfile/#using-environment-variables>.
- Parameters: <https://www.jenkins.io/doc/book/pipeline/syntax/#parameters>.
- Handling credentials in Pipeline:
<https://www.jenkins.io/doc/book/pipeline/jenkinsfile/#handling-credentials>.